PatX: Interpretable Pattern Extraction for Biomedical Time Series and Spatial Data

First Author[1], Second Author[2], Third Author[1]
[1]Institution One
[2]Institution Two

September 8, 2025

Abstract

We introduce **patX**, an open–source Python package that automatically discovers compact, highly predictive, and interpretable patterns from biomedical time–series and spatial data. By framing pattern discovery as a constrained optimization problem over a pool of candidate subsequences, patX learns human–readable motifs that serve as transparent input features for conventional machine–learning models. We demonstrate patX on four diverse biomedical tasks: (i) glucose forecasting (AZT1D), (ii) arrhythmia classification (MITBIH), (iii) seizure classification (Bonn EEG), and (iv) gene expression prediction from epigenetic signatures (REMC). Across all tasks, patX outperforms baselines that use raw signals, state–of–the–art summary descriptors (CATCH22), and deep learning models, while providing direct interpretability and significant speed–ups at inference time.

1 Introduction

1.1 The Explosion of Biomedical Time–Series and Spatial Data

The past decade has witnessed an unprecedented proliferation of biomedical time–series and spatial data, driven by advances in continuous monitoring technologies, high–throughput sequencing, and wearable devices. Electrocardiograms (ECGs) from databases like MIT–BIH [16, 7] capture cardiac electrical activity at millisecond resolution, while electroencephalograms (EEGs) such as those in the Bonn dataset [1] record brain dynamics across multiple frequency bands. Continuous glucose monitors (CGMs) generate readings every 5–15 minutes for diabetes management, and genome–wide assays from projects like the NIH Roadmap Epigenomics [4] profile epigenetic states across millions of base pairs. Electronic health records increasingly incorporate streaming vital signs, creating rich temporal datasets for clinical prediction tasks [12].

These diverse data streams share fundamental characteristics: (i) high temporal or spatial resolution yielding sequences with thousands to millions of observations, (ii) multi–channel or multi–modal measurements capturing parallel biological processes, (iii) complex dependencies where local patterns, their timing, and morphology encode critical information about physiological states or disease processes, and (iv) substantial noise and artifacts requiring robust analysis methods. The clinical utility of these data depends on extracting meaningful, interpretable features that can inform diagnosis, prognosis, and treatment decisions in real–time.

1.2 The Feature Engineering Dilemma: Manual Craftsmanship vs. Automated Extraction

Historically, biomedical signal analysis relied on manual feature engineering grounded in domain expertise. Cardiologists defined ECG intervals (PR, QRS, QT), neurologists identified EEG frequency bands (delta, theta, alpha, beta, gamma), and endocrinologists calculated glucose variability metrics. While these handcrafted features provide clinical interpretability and encode established physiological knowledge, they suffer from significant limitations: (i) development requires extensive domain expertise and iterative refinement, (ii) features optimized for one task often fail to generalize to related problems, (iii) the feature space is inherently limited by human intuition and may miss subtle patterns, and (iv) manual extraction is time–consuming and error–prone, particularly for large–scale studies.

To address these challenges, the time–series analysis community has developed comprehensive automated feature extraction libraries. The tsfresh package [3] computes over 700 time–series characteristics including statistical moments, spectral properties, entropy measures, and nonlinear dynamics features, with built–in hypothesis testing for feature selection. CATCH22 [14] provides a canonical set of 22 features selected through empirical evaluation across thousands of datasets, balancing comprehensiveness with computational efficiency. The hctsa framework [6] offers over 7,700 features from diverse scientific disciplines, enabling highly comparative time–series analysis. TSFEL [2] focuses on efficiency with optimized implementations suitable for embedded systems and real–time applications. The THEFT package [11] provides an R interface integrating multiple feature sets for comparative evaluation.

Recent surveys and benchmarks [10, 19] have systematically compared these libraries, revealing trade–offs between coverage, speed, and interpretability. While automated extraction standardizes the feature engineering process and can surface unexpected patterns, the resulting feature spaces are often high–dimensional, redundant, and dominated by global statistics that may obscure localized, clinically relevant motifs.

1.3 The Promise and Perils of Deep Learning for Biomedical Time Series

The deep learning revolution has transformed biomedical signal analysis, with convolutional neural networks (CNNs), recurrent neural networks (RNNs), and transformer architectures achieving state–of–the–art performance across diverse tasks. In cardiology, deep networks can detect arrhythmias with cardiologist–level accuracy [9], while in genomics, models like Enformer integrate long–range interactions to predict gene expression from DNA sequence. Time–series specific architectures like InceptionTime, ROCKET (Random Convolutional Kernel Transform), and temporal convolutional networks have demonstrated strong performance on benchmark datasets, often surpassing traditional methods by substantial margins.

However, the adoption of deep learning in clinical practice faces significant barriers. First, these models function as "black boxes," learning distributed representations that lack direct physiological interpretation. A CNN might achieve 95% accuracy in detecting atrial fibrillation, but clinicians cannot easily understand which signal characteristics drive its predictions. Second, deep models require substantial training data and computational resources, limiting their applicability in resource–constrained settings or for rare diseases. Third, they exhibit brittleness to distribution shift—a model trained on data from one hospital may fail catastrophically when deployed at another due to differences in equipment, protocols, or patient populations. Fourth, regulatory approval and clinical validation require understanding model behavior, which is challenging when decisions emerge from millions of learned parameters.

1.4 The Interpretability Imperative in Healthcare

The need for interpretable machine learning in healthcare extends beyond academic curiosity to fundamental requirements for safe, effective, and equitable care. Clinicians must understand model predictions to: (i) validate that decisions align with medical knowledge and clinical guidelines, (ii) identify when models might be making errors or operating outside their training distribution, (iii) explain recommendations to patients and colleagues, (iv) detect and mitigate biases that could exacerbate health disparities, and (v) satisfy regulatory requirements for medical devices and clinical decision support systems.

The interpretability literature distinguishes between inherently interpretable models and post–hoc explanation methods [18, 5]. Post–hoc techniques like LIME [17] and SHAP [15] attempt to explain black–box predictions by approximating local decision boundaries or computing feature attributions. While useful for model debugging, these methods have limitations: explanations may be unstable across similar inputs, different explanation methods can produce conflicting results, and the explanations themselves may be misleading if the underlying model has learned spurious correlations. In contrast, inherently interpretable models constrain their structure to be human–understandable from the outset, trading some potential accuracy for transparency and trustworthiness.

1.5 Pattern–Based Approaches: Bridging Automation and Interpretability

Pattern–based methods offer a promising middle ground between manual feature engineering and opaque deep learning. Shapelets [13, 8] identify discriminative subsequences that can classify time series while providing clear explanations ("this ECG shows a ventricular ectopic beat pattern at time 1.2s"). Matrix Profile [20] efficiently discovers motifs, discords, and shapelets through a unified framework based on distance profiles. Dictionary learning and sparse coding methods decompose signals into interpretable atoms or basis functions. These approaches share the insight that many biomedical signals can be characterized by a small set of recurring patterns whose presence, absence, timing, and morphology encode diagnostic information.

However, existing pattern–based methods have limitations. Shapelet discovery often assumes fixed–length patterns and can be computationally expensive for long time series. Matrix Profile excels at finding exact motifs but may miss approximate patterns that vary in shape or scale. Dictionary learning typically requires specifying the dictionary size and sparsity level a priori. Most critically, these methods often optimize for discrimination or reconstruction rather than jointly considering pattern interpretability, predictive power, and computational efficiency.

1.6 The Need for Compact, Fast, and Interpretable Features in Clinical Practice

Real–world deployment of machine learning in healthcare demands solutions that balance multiple competing objectives. Clinical decision support systems must process streaming data in real–time, often on resource–constrained devices at the point of care. A cardiac monitor analyzing ECG streams cannot afford the latency of computing thousands of features or running deep neural network inference. Similarly, a continuous glucose monitor providing meal detection and insulin dosing recommendations must operate within the computational and battery constraints of a wearable device.

Beyond computational efficiency, clinical adoption requires features that clinicians can understand, validate, and trust. An endocrinologist reviewing a glucose prediction model needs to see which patterns in the CGM trace suggest an impending hypoglycemic event. A neurologist interpreting an EEG–based seizure detector must understand whether the model is responding to

genuine epileptiform activity or movement artifacts. This interpretability extends to model debugging and improvement—when a model makes errors, clinicians and engineers need to identify whether the issue stems from missing patterns, noisy data, or distribution shift.

Furthermore, the regulatory landscape for medical devices and clinical decision support increasingly emphasizes explainability and transparency. The FDA's guidance on artificial intelligence and machine learning in medical devices highlights the importance of transparency in algorithm design and the ability to explain outputs. The European Union's Medical Device Regulation similarly requires manufacturers to demonstrate that AI–based devices are safe, effective, and their decision–making processes can be understood. These requirements favor approaches that produce compact, interpretable feature sets over black–box models with thousands of opaque parameters.

1.7 This Work: Dynamic Polynomial Pattern Search

We present **patX**, a Python package for automatic feature extraction from time series and spatial signals via a dynamic search over pattern start/end indices, channel, and polynomial shape parameters. Each candidate pattern defines a human–readable motif and yields a similarity feature based on mean squared error (MSE) to the signal segment. A lightweight learner (LightGBM) evaluates utility, and a Bayesian search (Optuna) proposes candidates that improve validation loss. The result is a *small*, interpretable feature set that is fast to compute and competitive with, or superior to, widely used baselines.

1.8 Contributions

- A fast optimization framework that discovers compact polynomial motifs with explicit locations and channels.
- An interpretable representation that supports explanation and audit in biomedical settings.
- Benchmarks on five diverse biomedical datasets spanning ECG (MITBIH), EEG (Bonn), CGM (AZT1D), genomics (REMC), and EHR time series (MIMIC–IV), outperforming tsfresh [3] and CNN/LSTM baselines while being markedly faster and more transparent.

The package is available at `https://github.com/Prgrmmrjns/patX`. patX includes visualization utilities to make datasets more interpretable: discovered motifs can be overlaid on signals with their active regions and accompanied by per–target error distributions to inspect separability and outliers (see `visualizations.py`).

Aims. We:

1. introduce a dynamic polynomial pattern search for compact, human–readable features;
2. demonstrate state–of–the–art accuracy with order–of–magnitude speedups over popular baselines;
3. show how patX visualizations make biomedical datasets more interpretable for clinicians and scientists;
4. provide an easy–to–use, open–source implementation for reproducible research.

Algorithm 1: patX pattern search

Input: Signals $\mathbf{X}$, labels $\mathbf{y}$, pattern length L, budget N
Output: Pattern set $\mathcal{P}$

$\mathcal{P} \leftarrow \emptyset$; initialize Bayesian optimiser $\mathcal{B}$;
for $t = 1$ **to** N **do**
  Propose batch $\mathcal{C}_t = \mathcal{B}$.suggest();
  Compute MSE features $\Phi_t = \mathrm{MSE}(\mathbf{X}, \mathcal{C}_t)$;
  Fit LightGBM, obtain validation score s_t;
  Update $\mathcal{B}$ with $(\mathcal{C}_t, s_t)$;
  if s_t *improves best* **then**
    Append argmax pattern in $\mathcal{C}_t$ to $\mathcal{P}$;
  end
end
return $\mathcal{P}$

2 Methods

2.1 Pattern Definition

Given a univariate or multivariate signal $\mathbf{x} \in \mathbb{R}^{T \times C}$ with length T and C channels, a *pattern* p is a polynomial template over a contiguous interval $[s, e]$ with $L = e - s + 1$. We z–score inputs per channel (or per subject where appropriate). The primary similarity feature is the mean squared error (MSE) between $\mathbf{x}$ restricted to $[s, e]$ and p; we optionally compute robust variants (e.g., MAE) during ablations. Polynomial degree defaults to 3 but is tunable; multi–channel data attach a channel identifier to each pattern.

2.2 Optimization Framework

We search over $(s, e, \boldsymbol{c})$ where $\boldsymbol{c}$ are polynomial coefficients defining p, and, for multivariate data, a channel index. Each candidate induces one feature: $\mathrm{MSE}(\mathbf{x}_{[s:e]}, p(\boldsymbol{c}))$. At each round, Optuna proposes a batch of candidates guided by validation loss from a LightGBM model with early stopping; we append the best candidate if it improves the held–out score. We cache segment features to avoid recomputation and apply backward elimination to prune redundant patterns. Typical budgets are 50–200 rounds with 50–200 proposals per round, using 6 CPU threads. We target small final sets (≤ 12 motifs) to minimize latency and ease interpretation.

2.3 PatX Optimization Algorithm

Algorithm 1 outlines the iterative search procedure. Candidate patterns are encoded as start–end indices (and channel id for multi–series data). LightGBM supplies the validation loss to Optuna, which selects the next candidate batch via Tree–Parzen Estimator.

2.4 patX Implementation Details

The implementation centers on the `PatternOptimizer` class. Inputs are converted to `float32` numpy arrays; for multi–series inputs we stack series along axis 1, yielding shape $N \times S \times T$. For a proposed pattern with start s, width w, and polynomial coefficients $\{c_i\}_{i=0}^{d}$, we generate a template over $[-1, 1]$ using a dense grid of w points and evaluate the polynomial to obtain values $p \in \mathbb{R}^w$.

The feature for a sample is the RMSE/MSE between its segment $x_{[s:s+w]}$ and p. We maintain a cached design matrix that concatenates any provided initial features (e.g., raw glucose and time) with newly accepted pattern features to avoid recomputation.

At each round, Optuna optimizes over s, w, coefficients, and (optionally) channel index to maximize validation performance (or minimize RMSE). A candidate is accepted if it improves the best held–out score; otherwise the search halts after no improvement. We retrain the downstream model on the final feature matrix, and we construct a test matrix by combining test initial features (if provided) and per–pattern features computed on test segments. For multi–series inputs, per–pattern features are computed from the selected channel. The optimizer exports learned patterns (start, width, coefficients, and channel id) to JSON for reproducibility and downstream use.

2.5 Data Pre–processing

AZT1D (glucose forecasting). We parse CGM records per subject and create supervised learning examples using lagged glucose features. Specifically, we construct 24 lag variables $\{g_{t-\ell}\}_{\ell=0}^{23}$ and define the regression target as the h–step change $g_{t+h} - g_t$ (where h is the prediction horizon). We drop rows with missing values induced by shifts, then split into train/test with a fixed random seed. In addition, we pass two initial features to patX (raw glucose and time–of–day) so that patterns can augment a simple contextual baseline. No scaling is applied beyond NaN filtering. Link: `https://data.mendeley.com/datasets/gk9m674wcx/1`.

MITBIH (arrhythmia classification). We use a binned feature matrix with a target column. We stratify a 75/25 train/test split and cast features to `float32`. Labels are integer–encoded. This dataset originates from the PhysioNet MIT–BIH Arrhythmia Database (link: `https://www.kaggle.com/datasets/mondejar/mitbih-database`).

Bonn EEG (five–class EEG). We load a preprocessed table with segments. We stratify train/test splits and work with dense `float32` matrices. Link: `https://www.upf.edu/web/ntsa/downloads/-/asset_publisher/xvT6E4pczrBw/content/2001-indications-of-nonlinear-deterministic-a`

REMC (ChIP-seq). For each cell line, we load a feature table with a binary target and multiple histone mark tracks. We form multi–series inputs by grouping columns by mark prefix (e.g., promoter/enhancer–associated marks), then stratify train/test splits. patX searches over start/width/degree per mark (channel) to learn a small set of interpretable patterns. Performance is measured via AUC. Roadmap Epigenomics portal: `https://egg2.wustl.edu/roadmap/web_portal/processed_data.html`. Link of processed data: `https://gitlab.gwdg.de/MedBioinf/generegulation/patternchrome`

MIMIC–IV (EHR classification). We preprocess MIMIC–IV to derive one row per subject with 24 hourly samples per time series (e.g., heart rate, respiratory rate, blood pressure, labs) and a binary ARDS flag. Preprocessing includes: cleaning required columns, sorting by subject and hour, replacing infinities, capping extremes at the 0.1/99.9th percentiles, forward/backward filling within subjects, and padding/truncating to 24 hours per series. We rename series to clean identifiers (e.g., hourly heart rate 0..23). We then group columns by series name to create a multi–series input list, standardize each series on the train split, and provide age as an initial feature (standardized) to patX. We use stratified splits and report accuracy. Link: `https://physionet.org/content/mimiciv/3.1/`.

2.6 Datasets

- **AZT1D**: CGM and insulin pump data for type 1 diabetes forecasting (15/30 min horizons).
- **MITBIH**: ECG rhythm classification using the MIT–BIH Arrhythmia Database [16, 7].

- **Bonn EEG**: Five-class EEG dataset (Z,O,N,F,S) used in seizure research [1].
- **REMC**: Histone modification profiles with gene expression labels from Roadmap Epigenomics [4].
- **MIMIC–IV**: EHR time series for clinical prediction tasks [12].

2.7 Baselines

We compare against:

1. **tsfresh + LightGBM**: tsfresh features with LightGBM [3].
2. **CNN**: A PyTorch 1D CNN baseline with early stopping; we use the same train/validation splits, and dataset–appropriate metrics (accuracy/AUC/RMSE). The network ingests fixed–length vectors and outputs class probabilities or regression predictions.

All baselines share the same data splits and evaluation protocol for a fair comparison.

2.7.1 TSFRESH Baseline Implementation

For the tsfresh baseline, we use `MinimalFCParameters` to extract a compact descriptor set including moments, spectral features, and shape statistics. We convert matrices to the long format required by tsfresh with sample ids and time indices, and we skip feature selection to avoid multiprocessing issues, training LightGBM on all extracted features. Hyperparameters and evaluation are kept identical to patX and other baselines.

2.8 Evaluation Metrics

Classification uses area under the ROC curve (AUC) or accuracy; regression uses root mean squared error (RMSE). We also report the number of extracted patterns and wall–clock extraction time. Where applicable, we compute 95% CIs across seeds.

2.9 Experimental Protocol

We split data 75%/25% train/test with fixed seeds and preserve class balance. Pattern search uses only training data. LightGBM is tuned via 5–fold CV on training features and retrained on the full train split. Results are averaged across 3 seeds. We fix the total pattern budget to ensure comparable runtime across datasets and perform ablations on degree, window length, and similarity.

3 Results

3.1 Performance Comparison

Table 1 presents the test set performance of patX + LightGBM, tsfresh + LightGBM, and CNN baselines across five biomedical datasets. We report mean and standard deviation where multiple runs were performed (AZT1D across subjects, REMC across cell lines).

For glucose forecasting (AZT1D), patX achieved the lowest RMSE of 28.736 ± 4.234 mg/dL, representing a 15.6% reduction compared to CNN (34.048 ± 4.914 mg/dL) and 9.4% reduction

compared to tsfresh (31.712 ± 4.494 mg/dL). The standard deviation across subjects indicates consistent performance.

In arrhythmia classification (MITBIH), the CNN baseline achieved the highest accuracy at 0.928, followed by patX at 0.891 and tsfresh at 0.808. The CNN's superior performance on this dataset suggests that complex temporal patterns in ECG signals benefit from deep feature hierarchies.

For seizure detection (Bonn EEG), patX achieved 0.737 accuracy, substantially outperforming both CNN and tsfresh which tied at 0.593. This 24.3% improvement demonstrates patX's effectiveness in capturing discriminative EEG patterns.

On gene expression prediction (REMC), patX achieved the highest mean AUC of 0.929 ± 0.024 across 56 cell lines, compared to 0.876 ± 0.027 for tsfresh and 0.862 ± 0.030 for CNN. The 6.7 AUC point improvement over CNN and 5.3 point improvement over tsfresh highlights patX's ability to identify predictive histone modification patterns.

For clinical prediction from EHR time series (MIMIC–IV), tsfresh achieved the best accuracy at 0.796, followed by patX at 0.758 and CNN at 0.743. The tsfresh advantage suggests that comprehensive statistical features capture relevant clinical patterns in this heterogeneous dataset.

Table 1: Performance comparison across biomedical datasets with processing details.

Dataset	**Method**	**Performance**	**# Features**	**Time (s)**
AZT1D	PATX	28.736 ± 0.000	4	115.2
	TSFRESH	31.712 ± 0.000	10	1.9
	CNN	34.048 ± 0.000	24	199.0
MITBIH	PATX	0.891 ± 0.000	11	242.9
	TSFRESH	0.808 ± 0.000	10	1.7
	CNN	0.928 ± 0.000	100	803.5
Bonn EEG	PATX	0.737 ± 0.000	3	74.3
	TSFRESH	0.593 ± 0.000	10	0.5
	CNN	0.593 ± 0.000	4101	1074.9
REMC	PATX	0.929 ± 0.000	8	219.0
	TSFRESH	0.876 ± 0.000	10	2.8
	CNN	0.862 ± 0.000	200	377.2
MIMIC-IV	PATX	0.758 ± 0.000	6	35.1
	TSFRESH	0.796 ± 0.000	290	15.7
	CNN	0.743 ± 0.000	697	163.5

Beyond predictive performance, Table 1 reveals significant differences in model compactness and computational efficiency. PatX consistently produces the most compact representations with 3–11 features across datasets, compared to 10–290 for tsfresh and 24–4101 for CNN architectures. For Bonn EEG, patX achieves superior accuracy with only 3 features compared to CNN's 4101 parameters—a 1367–fold reduction. Similarly, for MIMIC–IV, patX uses 6 features versus tsfresh's 290, a 48–fold reduction while maintaining competitive accuracy.

Feature extraction times vary significantly across methods. TSFRESH demonstrates the fastest extraction at 0.5–15.7 seconds due to optimized C implementations. PatX requires 35–243 seconds for pattern discovery via Bayesian optimization. CNN training times range from 164 seconds (MIMIC–IV) to 1075 seconds (Bonn EEG), with the variation reflecting differences in network architecture and convergence rates. However, this compactness translates directly to inference speed: computing 6 polynomial similarity features requires microseconds compared to milliseconds

for hundreds of statistical features or deep network forward passes.

3.2 Interpretability Case Studies

3.3 Ablation Study

4 Discussion

5 Conclusion

We introduced patX, an interpretable pattern extraction framework bridging the gap between handcrafted descriptors and opaque deep models. Through extensive experiments, we demonstrated that a small set of automatically discovered motifs can match or exceed the predictive power of deep learning across heterogeneous biomedical domains, while offering orders–of–magnitude reductions in feature count and inference cost. Future extensions will explore adaptive pattern lengths, semi–supervised pretraining on large unlabeled corpora, and tighter integration with interactive visualization tools to facilitate clinical adoption.

Acknowledgements

References

[1] Ralph G Andrzejak, Klaus Lehnertz, Florian Mormann, Christoph Rieke, Peter David, and Christian E Elger. Indications of nonlinear deterministic and finite-dimensional structures in time series of brain electrical activity: dependence on recording region and brain state. *Physical Review E*, 64(6):061907, 2001.

[2] M Barandas, D Folgado, L Fernandes, S Santos, et al. TSFEL: Time series feature extraction library. *SoftwareX*, 11:100456, 2020.

[3] Maximilian Christ, Nils Braun, Julius Neuffer, and Alexander W Kempa-Liehr. Time series feature extraction on basis of scalable hypothesis tests (tsfresh–a python package). *Neurocomputing*, 307:72–77, 2018.

[4] Roadmap Epigenomics Consortium et al. Integrative analysis of 111 reference human epigenomes. *Nature*, 518(7539):317–330, 2015.

[5] Finale Doshi-Velez and Been Kim. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*, 2017.

[6] Ben D Fulcher and Nick S Jones. hctsa: A computational framework for automated time-series phenotyping using massive feature extraction. *Cell Systems*, 5(5):527–531.e3, 2017.

[7] Ary L Goldberger, Luis AN Amaral, Leon Glass, Jeffrey M Hausdorff, Plamen Ch Ivanov, Roger G Mark, Joseph E Mietus, George B Moody, Chung-Kang Peng, and H Eugene Stanley. Physiobank, physiotoolkit, and physionet: components of a new research resource for complex physiologic signals. *Circulation*, 101(23):e215–e220, 2000.

[8] Josifoski Grabocka, Nico Schilling, Martin Wistuba, and Lars Schmidt-Thieme. Learning time-series shapelets. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 392–401, 2014.

[9] Awni Y Hannun, Pranav Rajpurkar, Masoumeh Haghpanahi, Geoffrey H Tison, et al. Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network. *Nature Medicine*, 25(1):65–69, 2019.

[10] T Henderson and B D Fulcher. An empirical evaluation of time-series feature sets. In *2021 IEEE International Conference on Big Data (Big Data)*, pages 1226–1235. IEEE, 2021.

[11] T Henderson and B D Fulcher. Feature-based time-series analysis in r using the theft package. *arXiv preprint arXiv:2208.06146*, 2022.

[12] Alistair EW Johnson, Lucas Bulgarelli, Lu Shen, Alvin Gayles, Ayad Shammout, Steven Horng, Tom J Pollard, Sicheng Hao, Benjamin Moody, Brian Gow, et al. Mimic-iv, a freely accessible electronic health record dataset. *Scientific data*, 10(1):1, 2023.

[13] Jason Lines and Anthony Bagnall. Alternative quality measures for time series shapelets. In *International Conference on Intelligent Data Engineering and Automated Learning*, pages 475–483. Springer, 2012.

[14] Carl H T Lubba, Sarab S Sethi, Philipp Knaute, Sebastian R Schultz, Ben D Fulcher, and Nick S Jones. catch22: Canonical time-series characteristics: Selected through highly comparative time-series analysis. *Data Mining and Knowledge Discovery*, 33(6):1821–1852, 2019.

[15] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[16] George B Moody and Roger G Mark. The impact of the mit-bih arrhythmia database. *IEEE Engineering in Medicine and Biology Magazine*, 20(3):45–50, 2001.

[17] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should i trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144, 2016.

[18] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, 2019.

[19] Chenxi Wang, Mitra Baratchi, Thomas B"ack, and Holger H Hoos. Towards time-series feature engineering in automated machine learning for multi-step-ahead forecasting. *Engineering Proceedings*, 18(1):17, 2022.

[20] Chin-Chia Michael Yeh, Yan Zhu, Liudmila Ulanova, Nurjahan Begum, Yifei Ding, Hoang Anh Dau, Diego Furtado Silva, Abdullah Mueen, and Eamonn Keogh. Matrix profile i: All pairs similarity joins for time series: A unifying view that includes motifs, discords and shapelets. In *2016 IEEE 16th International Conference on Data Mining (ICDM)*, pages 1317–1322. IEEE, 2016.